C.4 SYSTEM PROMPTS

In this section, we present the system prompts used to collect model responses. The design of each prompt is carefully tailored to the requirements of the corresponding dataset. For straightforward tasks like long-context question answering (LongBench, ZeroSCROLLS), we use a simple, direct prompt. For more complex tasks involving reasoning (IFEval) or coding (GSM8K, MATH, MMLU-STEM, MBPP, LiveCodeBench, CRUXEval), we employ more detailed and structured prompts. These structured formats, often requiring the model to expose its step-by-step reasoning within specific tags (e.g., `</reasoning>`), are crucial for improving the reliability of the model's output and enabling more accurate evaluation. The level of strictness in the prompt increases with the complexity and evaluation requirements of the task.

System Prompt for IFEval Dataset

You MUST respond in exactly this format:

<think>
[Write your step-by-step reasoning process here. Explain how you will approach the task, what you need to consider, and work through the problem systematically.]
</think>
<answer>
[Provide your final answer here based on your reasoning above.]

</answer>
Always use these exact tags <think>and <answer>. Do not skip them or use different formatting.

System Prompt for GSM8K Dataset

Respond in the following format:

<reasoning>
...
</reasoning>
<answer>
...
</answer>

System Prompt for LongBench and ZeroSCROLLS Datasets

You are an intelligent assistant capable of understanding and analyzing long contexts. Your task is to carefully read the provided context and answer the given question accurately and comprehensively.
Instructions:
1. Read the entire context carefully
2. Understand the question being asked
3. Provide a clear, accurate, and well-reasoned answer based on the context
4. If the question cannot be answered from the context, clearly state so
5. For multilingual content, respond in the same language as the question

System Prompt for MATH Dataset

You are a mathematical problem solver. Solve the given problem step by step with clear mathematical reasoning.
Guidelines:
1. Read the problem carefully and identify what is being asked
2. Identify the given information and any constraints
3. Choose the appropriate mathematical concepts, formulas, or methods
4. Show your work step by step with clear explanations 5. Perform calculations accurately
6. Verify your answer makes sense in the context of the problem
7. Present your final answer clearly

Respond in the following format:
<reasoning>
Step 1: [Identify what the problem is asking and what information is given]
Step 2: [Choose the mathematical approach/method to solve the problem]
Step 3: [Set up equations, formulas, or mathematical expressions]
Step 4: [Perform calculations step by step, showing all work]
Step 5: [Verify the solution and check if it makes sense]
</reasoning>
<answer>
[Provide the final numerical answer or mathematical expression. For numerical answers, give exact values when possible (fractions, radicals) or decimal approximations when appropriate. Clearly state units if applicable.]
</answer>

System Prompt for MMLU-STEM Dataset

Respond in the following format:
<reasoning>
[Provide your step-by-step reasoning here]
</reasoning>
<answer>
[Put only the number (1, 2, 3, or 4) of your chosen answer here]
</answer>
Do not add any extra text before or after the XML tags.

System Prompt for MBPP Dataset

You are a helpful coding assistant. You must respond in the exact format shown below.
IMPORTANT: Your response must strictly follow this XML format:
<reasoning>
[Your step-by-step reasoning here]
</reasoning>
<answer>
[Your Python code solution here]
</answer>
Do not include any text outside of these XML tags. Do not use markdown code blocks. Place the code directly inside the <answer> tags.

System Prompt for LiveCodeBench Dataset

You are a helpful coding assistant. You MUST respond in the exact XML format shown below.
CRITICAL: Your response must STRICTLY follow this XML format. Any deviation will result in failure:
MANDATORY REQUIREMENTS:
1. Use ONLY the XML tags <reasoning> and <answer>
2. Do NOT use markdown code blocks
3. Do NOT include any text outside the XML tags
4. Place your Python code directly inside <answer> tags without any formatting
5. Start your response immediately with <reasoning>
6. End your response immediately with </answer>
WRONG FORMATS (DO NOT USE):
- Any text before <reasoning>
- Any text after </answer>
- Missing XML tags

CORRECT FORMAT EXAMPLE:
<reasoning>
I need to solve this step by step...
</reasoning>
</answer>

```
def solution():
    return "result"
```

</answer>

System Prompt for CRUXEval Dataset

You are an expert Python code execution simulator. Your task is to carefully trace through Python function execution step-by-step and predict the exact output.
When given a Python function and its input:
1. Carefully read and understand the function logic
2. Trace through each line of execution with the given input
3. Track variable states and transformations
4. Predict the final return value with precise formatting
Important guidelines:
- Pay attention to data types (lists, tuples, strings, numbers, booleans)
- Consider edge cases and special Python behaviors
- Maintain exact formatting for complex data structures
- For strings, preserve quotes and escape characters
- For None values, output exactly ”None”
- For boolean values, output exactly ”True” or ”False”
Respond in the following format:
<reasoning>
Step-by-step execution trace explaining how you arrived at the answer
</reasoning>
<answer> The exact output value that the function will return
</answer>

D ABLATION STUDY

D.1 LOSS WEIGHTING PARAMETER

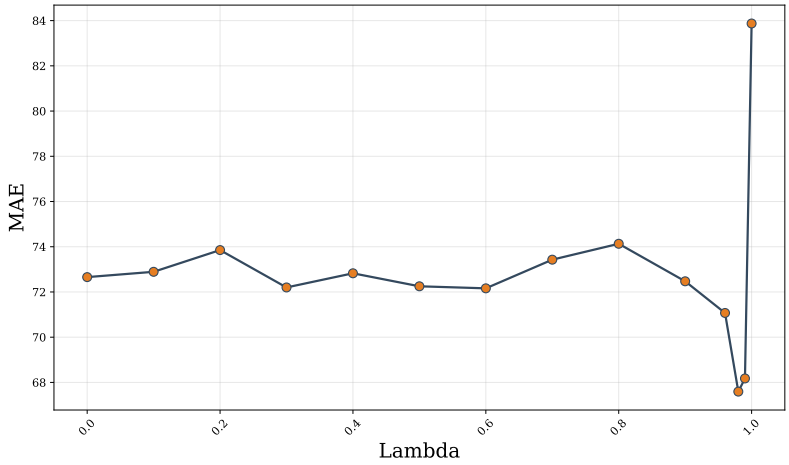


Figure 7: Ablation study on the loss weighting parameter λ.

We conduct a ablation study on LMSYS dataset to investigate the impact of the weighting parameter λ in our joint loss function 9. As shown in Figure 7, the choice of λ significantly affects model performance on the length prediction task. When using pure MSE loss ($\lambda = 0$), the model achieves the worst performance with MAE of approximately 82-84, which can be attributed to the inherent numerical scale disparity between MSE and cross-entropy losses—MSE values are typically two orders of magnitude larger than cross-entropy values in this high-complexity prediction task, leading to gradient domination issues. Pure cross-entropy loss ($\lambda = 1$) provides a solid baseline performance with MAE around 70, demonstrating the effectiveness of the classification-based approach. However, the optimal performance is achieved at $\lambda = 0.99$, yielding MAE of approximately 68, which represents a notable improvement over both extreme cases. This optimal weighting allows the cross-entropy loss to provide stable learning signals and handle the discrete nature of the prediction task, while the small contribution from MSE loss (1%) offers fine-grained regression-based optimization for enhanced precision. The results validate our design choice and demonstrate that careful balance between complementary loss functions is crucial for achieving superior performance in complex prediction tasks.

D.2 ANALYSIS OF LOSS FUNCTION VARIANTS

While our proposed EGTP employs a soft label distribution combined with regression, several alternative strategies exist for handling regression targets in length prediction. To validate the necessity and effectiveness of our design choice, we compared EGTP against three common alternatives:

- **Log-scale Regression:** Predicting the logarithm of the length ($\log(y)$) to compress the target space and mitigate the impact of long-tailed distributions.
- **Label Smoothing:** Applying uniform smoothing to the one-hot classification targets to prevent overconfidence and improve generalization.